

COFFEE SHOP PYSPARK PROJECT

Code Walkthrough — script.py Explained Section by Section

Bronze → Silver → Gold → Analytics

Companion to [Coffee_Shop_PySpark_Project_Report.pdf](#) and [script.py](#)

Prepared by Youssef — Big Data Engineering Interview Prep

How to Read This Document

This PDF walks through **every block of script.py** in the order it executes. Each block shows the explanation first, then the exact code. Comment lines from the script are highlighted in gold inside the code panels — in this project every line is commented for a reason: this file is meant to be read as interview-prep material, not just run.

For the business findings (charts, revenue numbers, rankings), see the companion report **Coffee_Shop_PySpark_Project_Report.pdf**. This document is purely about the *code* — what each API call does and why it was chosen.

0 · Imports & Configuration

Imports

Standard PySpark building blocks: `SparkSession` is the entry point; functions `as F` gives the vectorized, Catalyst-optimized column expressions; `Window` builds `OVER(PARTITION BY...ORDER BY...)` specs; `types` lets us declare an explicit schema instead of relying on (slower, riskier) inference.

```
import os
import shutil

from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window
from pyspark.sql.types import (
    StructType, StructField, IntegerType, StringType, DoubleType, DateType
)

import pandas as pd
import matplotlib
matplotlib.use("Agg") # headless backend - we only save PNGs, never show()
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
```

Project paths & color palette

Paths are computed relative to the script's own location so the project is portable. A single COLORS dict and matplotlib rcParams block keep every chart visually consistent across the script, notebook, and PDF report.

```
BASE_DIR      = os.path.dirname(os.path.abspath(__file__))
RAW_CSV_PATH  = os.path.join(BASE_DIR, "data", "coffee_shop.csv")
GOLD_DIR      = os.path.join(BASE_DIR, "outputs", "gold")
CHARTS_DIR    = os.path.join(BASE_DIR, "outputs", "charts")
SUMMARY_DIR   = os.path.join(BASE_DIR, "outputs", "summaries")

os.makedirs(CHARTS_DIR, exist_ok=True)
os.makedirs(SUMMARY_DIR, exist_ok=True)

COLORS = {
    "coffee_dark": "#4B2E2B",
    "coffee":      "#6F4E37",
    "coffee_light": "#A9746E",
    "cream":        "#E8DCCA",
    "gold":         "#C89B3C",
    "green":        "#5B8266",
    "red":          "#B33A3A",
}
```

1 · Spark Session

Creating the SparkSession

SparkSession.builder

`.master("local[*]")` runs Spark locally using every available core — appropriate for a ~150K-row dataset. On a real cluster this line is simply omitted and the master comes from `spark-submit` / YARN / Kubernetes instead. `spark.sql.shuffle.partitions` is lowered from Spark's default of 200 because 200 tiny shuffle partitions on a small dataset just adds scheduling overhead.

```
spark = (
    SparkSession.builder
        .appName("CoffeeShopSalesAnalytics")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .config("spark.driver.memory", "4g")
        .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR") # keep console output readable
```

2 · Bronze Layer — Raw Ingestion

Explicit schema

StructType / StructField

An explicit StructType is declared instead of `inferSchema=True`. Schema inference forces Spark to make an extra pass over the file just to guess column types — and it can guess wrong. Declaring the schema up front is faster *and* safer, which is why it's standard in production ingestion code.

```
bronze_schema = StructType([
    StructField("transaction_id", IntegerType(), True),
    StructField("transaction_date", StringType(), True), # parsed in SILVER
    StructField("transaction_time", StringType(), True), # parsed in SILVER
    StructField("store_id", IntegerType(), True),
    StructField("store_location", StringType(), True),
    StructField("product_id", IntegerType(), True),
    StructField("transaction_qty", IntegerType(), True),
    StructField("unit_price", DoubleType(), True),
    StructField("product_category", StringType(), True),
    StructField("product_type", StringType(), True),
    StructField("product_detail", StringType(), True),
    StructField("Size", StringType(), True),
    StructField("Total Bill", DoubleType(), True),
    StructField("Month Name", StringType(), True),
    StructField("Day Name", StringType(), True),
    StructField("Hour", IntegerType(), True),
    StructField("Day of Week", IntegerType(), True),
    StructField("Month", IntegerType(), True),
])
```

Reading the CSV

spark.read.csv

`.option("header", True)` skips the header row; the explicit schema is attached with `.schema(...)` instead of letting Spark infer it.

```
bronze_df = (
    spark.read
    .option("header", True)
    .option("encoding", "UTF-8")
    .schema(bronze_schema)
    .csv(RAW_CSV_PATH)
)

bronze_count = bronze_df.count()
print(f"BRONZE: loaded {bronze_count:,} raw rows")
```

3 · Silver Layer — Cleaning & Business Rules

Renaming & trimming

withColumnRenamed / F.trim

Spark technically allows spaces in column names, but they're painful in SQL expressions, so every spaced column is renamed to snake_case with `withColumnRenamed`. Every string column is also trimmed with `F.trim()` — a very common real-world data-quality issue (trailing/leading whitespace from CSV exports).

```
silver_df = (
    bronze_df
    .withColumnRenamed("Total Bill", "total_bill_source")
    .withColumnRenamed("Size", "size")
    .withColumnRenamed("Month Name", "month_name")
    .withColumnRenamed("Day Name", "day_name")
    .withColumnRenamed("Hour", "hour_of_day")
    .withColumnRenamed("Day of Week", "day_of_week")
    .withColumnRenamed("Month", "month_num")
    .withColumn("store_location", F.trim(F.col("store_location")))
    .withColumn("product_category", F.trim(F.col("product_category")))
    .withColumn("product_type", F.trim(F.col("product_type")))
    .withColumn("product_detail", F.trim(F.col("product_detail")))
    .withColumn("size", F.trim(F.col("size")))
```

Parsing dates & timestamps

F.to_date / F.to_timestamp

The raw date/time are plain strings ("6/1/2023", "11:33:29 AM"). `F.to_date` and `F.to_timestamp` parse them into real Spark date/timestamp types using explicit format patterns — never rely on default parsing in a real project, since format ambiguity (US vs. international date order) is a classic source of silent bugs.

```
.withColumn("transaction_date", F.to_date(F.col("transaction_date"), "M/d/yyyy"))
.withColumn("transaction_ts",
    F.to_timestamp(
        F.concat_ws(" ", F.col("transaction_date").cast("string"),
            F.col("transaction_time")),
        "yyyy-MM-dd h:mm:ss a"))
```

Data-quality cross-check

F.round / F.abs

Rather than trusting the source `Total Bill` column, revenue is **recomputed independently** (`qty × unit_price`) and a boolean flag marks any row where the two disagree by more than a cent. This is a classic silver-layer data-quality gate: never trust a derived column from the source without verifying it.

```
.withColumn("total_amount",
    F.round(F.col("transaction_qty") * F.col("unit_price"), 2))
.withColumn("revenue_mismatch_flag",
    F.abs(F.col("total_amount") - F.col("total_bill_source")) > 0.01)
.drop("total_bill_source")
```

Filtering & de-duplication

filter / dropDuplicates / cache

Structurally invalid rows (zero/negative quantity or price) are filtered out, then exact duplicate transactions are dropped by their natural key with `dropDuplicates`. `.cache()` is called because `silver_df` is reused by every downstream step — without caching, Spark would recompute the whole chain from the CSV every single time it's referenced.

```
.filter((F.col("transaction_qty") > 0) & (F.col("unit_price") > 0))
.dropDuplicates(["transaction_id"])
)

silver_df.cache() # reused by every downstream step below
silver_count = silver_df.count()
mismatches = silver_df.filter(F.col("revenue_mismatch_flag")).count()
```

4 · Gold Layer — Star Schema

dim_date

date_id surrogate key

Built from the distinct dates present in the data plus calendar attributes needed for time-series analytics. `date_id` is an integer surrogate key in `yyyyMMdd` form — a common, sort-friendly convention for date dimensions.

```
dim_date = (
    silver_df
    .select("transaction_date", "month_name", "day_name",
           "day_of_week", "month_num")
    .distinct()
    .withColumn("date_id", F.date_format("transaction_date", "yyyyMMdd").cast("int"))
    .withColumn("quarter", F.quarter("transaction_date"))
    .withColumn("is_weekend", F.col("day_name").isin("Saturday", "Sunday"))
    .select("date_id", F.col("transaction_date").alias("full_date"),
           "day_name", "day_of_week", "month_name", "month_num",
           "quarter", "is_weekend")
    .orderBy("date_id")
)
```

dim_store & dim_product

Both dimensions are simply the distinct combinations of their descriptive columns — in this dataset, `store_id` and `product_id` already act as natural/surrogate keys, so no extra key-generation step is needed.

```
dim_store = (
    silver_df
    .select("store_id", "store_location")
    .distinct()
    .orderBy("store_id")
)

dim_product = (
    silver_df
    .select("product_id", "product_category", "product_type",
           "product_detail", "size", "unit_price")
    .distinct()
    .orderBy("product_id")
)
```

fact_sales

One row per original transaction line: the numeric measures (transaction_qty, unit_price, total_amount) plus foreign keys into the three dimensions.

```
fact_sales = (
    silver_df
    .withColumn("date_id", F.date_format("transaction_date", "yyyyMMdd").cast("int"))
    .select(
        "transaction_id", "date_id", "store_id", "product_id",
        "hour_of_day", "transaction_qty", "unit_price", "total_amount"
    )
)
```

Writing partitioned Parquet

partitionBy + Parquet

Before writing, fact_sales is joined to a slice of dim_date using F.broadcast() just to pull in month_num for partitioning. Parquet is columnar, compressed, and splittable; partitioning fact_sales by month lets downstream Spark jobs or BI tools skip irrelevant partitions entirely on a month filter (partition pruning).

```
fact_with_month = (
    fact_sales.join(F.broadcast(dim_date.select("date_id", "month_num")),
        on="date_id", how="left")
)

dim_date.write.mode("overwrite").parquet(os.path.join(GOLD_DIR, "dim_date"))
dim_store.write.mode("overwrite").parquet(os.path.join(GOLD_DIR, "dim_store"))
dim_product.write.mode("overwrite").parquet(os.path.join(GOLD_DIR, "dim_product"))
(fact_with_month
 .write.mode("overwrite")
 .partitionBy("month_num")
 .parquet(os.path.join(GOLD_DIR, "fact_sales")))
```

5 · Analytics — Window Functions & Broadcast Joins

Building the analytics view with broadcast joins

F.broadcast()

The fact table is joined to all three dimensions using F.broadcast(). Since the dimensions are tiny (3 stores, 98 products, 181 dates) against a ~149K-row fact table, broadcasting ships a full copy of each small table to every executor instead of shuffling the huge fact table across the network — the standard optimization for a "big fact, small dimension" join. .drop("month_num") avoids a duplicate column since dim_date also carries month_num.

```
sales = (
    fact_with_month
    .drop("month_num")
    .join(F.broadcast(dim_store), on="store_id", how="left")
    .join(F.broadcast(dim_product), on="product_id", how="left")
    .join(F.broadcast(dim_date), on="date_id", how="left")
)
sales.cache()
sales.count() # materialize the cache
```

5a. Running cumulative revenue per store

Window.rowsBetween

A window function with an **unbounded-preceding frame** computes each store's running total month by month — the PySpark equivalent of SQL's `SUM(...) OVER (PARTITION BY store ORDER BY month ROWS UNBOUNDED PRECEDING)`.

```
store_month_revenue = (
    sales.groupBy("store_id", "store_location", "month_num", "month_name")
    .agg(F.round(F.sum("total_amount"), 2).alias("monthly_revenue"))
)

running_window = (
    Window.partitionBy("store_id")
    .orderBy("month_num")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
)

store_month_revenue = store_month_revenue.withColumn(
    "running_cumulative_revenue",
    F.round(F.sum("monthly_revenue").over(running_window), 2)
).orderBy("store_id", "month_num")
```

5b. Month-over-month growth with LAG()

F.lag()

`F.lag()` reaches back to the previous row of an ordered window (here, the prior month's revenue) so growth % can be computed in one pass — no self-join required.

```
total_by_month = (
    sales.groupBy("month_num", "month_name")
    .agg(F.round(F.sum("total_amount"), 2).alias("total_revenue"))
    .orderBy("month_num")
)

mom_window = Window.orderBy("month_num")
total_by_month = (
    total_by_month
    .withColumn("prev_month_revenue", F.lag("total_revenue").over(mom_window))
    .withColumn(
        "mom_growth_pct",
        F.round(
            (F.col("total_revenue") - F.col("prev_month_revenue"))
            / F.col("prev_month_revenue") * 100, 2
        )
    )
)
```


5c. Top-3 products per category with DENSE_RANK()

F.dense_rank()

`dense_rank()` over a window partitioned by category and ordered by revenue ranks products within each group without gaps; filtering to rank ≤ 3 is the standard "top-N per group" pattern — far more efficient than a Python loop per category.

```
category_product_revenue = (
    sales.groupBy("product_category", "product_type")
    .agg(F.round(F.sum("total_amount"), 2).alias("revenue"),
        F.sum("transaction_qty").alias("units_sold"))
)
rank_window = Window.partitionBy("product_category").orderBy(F.desc("revenue"))
top_products_per_category = (
    category_product_revenue
    .withColumn("rank_in_category", F.dense_rank().over(rank_window))
    .filter(F.col("rank_in_category") <= 3)
    .orderBy("product_category", "rank_in_category")
)
```

5d. Store performance ranking with RANK()

F.rank()

A simple global ranking (no `PARTITION BY`) orders stores by total revenue. `RANK()` (vs. `DENSE_RANK()`) leaves gaps after ties — the right choice when the exact competitive position matters.

```
store_performance = (
    sales.groupBy("store_id", "store_location")
    .agg(
        F.round(F.sum("total_amount"), 2).alias("total_revenue"),
        F.count("transaction_id").alias("total_transactions"),
        F.round(F.avg("total_amount"), 2).alias("avg_basket_value"),
    )
    .withColumn("revenue_rank", F.rank().over(Window.orderBy(F.desc("total_revenue"))))
    .orderBy("revenue_rank")
)
```

5e. Hourly & day-of-week demand patterns

groupBy + orderBy

Plain `groupBy` aggregations reveal peak ordering hours. Day names are reindexed in pandas afterward (Mon→Sun) since a default group-by would sort them alphabetically.

```
hourly_pattern = (
    sales.groupBy("hour_of_day")
    .agg(F.count("transaction_id").alias("num_transactions"),
        F.round(F.sum("total_amount"), 2).alias("revenue"))
    .orderBy("hour_of_day")
)

day_order = ["Monday", "Tuesday", "Wednesday", "Thursday",
             "Friday", "Saturday", "Sunday"]
dow_pattern = (
    sales.groupBy("day_name")
    .agg(F.round(F.sum("total_amount"), 2).alias("revenue"))
)
pdf_dow = (dow_pattern.toPandas()
    .set_index("day_name").reindex(day_order).reset_index())
```

5f. Category revenue share

`F.lit() + .collect()`

Total revenue is collected as a scalar with `.collect()[0][0]` (safe here since it's a single aggregated number, not a large result set), then used to compute each category's percentage of the whole.

```
category_share = (
    sales.groupBy("product_category")
        .agg(F.round(F.sum("total_amount"), 2).alias("revenue"))
)
total_revenue_all = category_share.agg(F.sum("revenue")).collect()[0][0]
category_share = (
    category_share
    .withColumn("pct_of_total",
        F.round(F.col("revenue") / F.lit(total_revenue_all) * 100, 2))
    .orderBy(F.desc("revenue"))
)
```

5g. Size mix within each category

`Broadcast self-join`

A small broadcast self-join against per-category totals converts raw unit counts into within-category percentages — the same "big table, tiny lookup" broadcast pattern used throughout section 5, just applied to an aggregated result instead of the raw fact table.

```
size_mix = (
    sales.filter(F.col("size").isNotNull())
        .groupBy("product_category", "size")
        .agg(F.sum("transaction_qty").alias("units_sold"))
)
cat_totals = (size_mix.groupBy("product_category")
    .agg(F.sum("units_sold").alias("cat_total")))
size_mix = (
    size_mix.join(F.broadcast(cat_totals), on="product_category")
    .withColumn("pct_within_category",
        F.round(F.col("units_sold") / F.col("cat_total") * 100, 1))
    .orderBy("product_category", F.desc("units_sold"))
)
```

6 · Wrap-Up

Cleanup

`unpersist() / stop()`

Cached DataFrames are explicitly unpersisted and the SparkSession is stopped — good practice in any standalone script so executor memory is released cleanly rather than left to garbage collection.

```
silver_df.unpersist()
sales.unpersist()
spark.stop()
```

Quick Reference — Every Technique in This Script

Technique	PySpark API	Used In
Explicit schema	StructType / StructField	Bronze

Technique	PySpark API	Used In
Date/time parsing	<code>F.to_date / F.to_timestamp</code>	Silver
De-duplication	<code>dropDuplicates()</code>	Silver
Caching	<code>.cache() / .unpersist()</code>	Silver, Analytics
Star schema + partitioning	<code>partitionBy().parquet()</code>	Gold
Broadcast join	<code>F.broadcast()</code>	Gold, Analytics (5a–5g)
Running total	<code>window.rowsBetween()</code>	5a
Prior-row comparison	<code>F.lag()</code>	5b
Top-N per group	<code>F.dense_rank()</code>	5c
Global ranking	<code>F.rank()</code>	5d
Group aggregation	<code>groupBy().agg()</code>	5e, 5f, 5g

Full runnable source: *script.py*. Executed version with rendered outputs: *Coffee_Shop_PySpark_Analytics.ipynb*.
 Business findings: *Coffee_Shop_PySpark_Project_Report.pdf*.